

웹공학 캡스톤디자인 프로젝트 제안서

수행 학기	2026학년도 1학기					
프로젝트 명	Trace : 콘텐츠 큐레이션 기반 개발자 성장 통합 플랫폼					
팀명	데픽					
구분	이름	학번	소속		전화번호	이메일
			1트랙	2트랙		
팀장	김홍근	2191049	웹공학트랙	빅데이터트랙	010-9859-3459	pentel2@naver.com
팀원	홍보민	2371354	모바일소프트웨어트랙	웹공학트랙	010-4577-8321	hwoochh@hansung.ac.kr
	박하영	2371324	모바일소프트웨어트랙	웹공학트랙	010-8919-3917	2371324@hansung.ac.kr
	조수현	2371358	빅데이터트랙	웹공학트랙	010-3551-1849	dreasm12345@hansung.ac.kr
지도교수	교과목명		웹공학 캡스톤디자인		과목코드	V024008-A
	성명		신성			
	소속		빅데이터트랙			
프로젝트 URL	https://github.com/Devpick-Org					

목 차

1. 프로젝트 수행 목적

- 1.1 프로젝트 정의
- 1.2 프로젝트 배경
 - 1.2 가 개발자 학습 생태계의 문제
 - 1.2 나 기존 LLM의 기술 최신성 한계
 - 1.2 다 학습-취업 연계 부재
 - 1.2 라 타겟 사용자 선정 근거
- 1.3 프로젝트 목표

2. 프로젝트 개요

- 2.1 프로젝트 설명
- 2.2 프로젝트 구조
- 2.3 시나리오
- 2.4 기대효과
- 2.5 제약조건
- 2.6 관련기술
- 2.7 개발도구

3. 프로젝트 추진 체계 및 일정

- 3.1 역할 분담
- 3.2 작업 흐름도
- 3.3 개발 일정

4. 참고자료

1. 프로젝트 수행 목적

1.1 프로젝트 정의

Trace는 개발자의 학습 전 과정을 하나의 흐름으로 연결하는 콘텐츠 큐레이션 기반 개발자 성장 플랫폼이다.

가. 플랫폼 개요

현재 개발자들은 기술 학습과 취업 준비를 위해 평균 5개 이상의 플랫폼을 오가고 있다. Stack Overflow에서 오류를 검색하고, Velog와 Medium에서 기술 블로그를 읽으며, GitHub에서 오픈소스 코드를 참고하고, YouTube에서 강의를 시청하며, 사람인과 잡코리아에서 채용 공고를 확인하는 식이다. 각 플랫폼에서 얻은 정보는 서로 연결되지 않고 분산된 채로 남는다.

Trace는 이 분산된 개발자 활동을 단일 플랫폼에서 처리할 수 있도록 통합한다. Stack Overflow, Velog, GitHub Trending 등에서 수집된 최신 기술 콘텐츠를 피드로 제공하고, AI가 사용자 레벨에 맞게 요약하며, 생성된 질문과 답변을 커뮤니티와 연계하고, 학습 이력이 자동으로 이력서와 면접 준비로 이어지는 end-to-end 흐름을 구현한다.

나. 핵심 기능 구성

Trace의 기능은 크게 6가지 영역으로 구성된다.

- **콘텐츠 탐색 및 AI 요약**

Stack Overflow, Velog, GitHub Trending, Hacker News, 국내 기술 블로그(Kakao, Naver D2, Toss 등)에서 1~6시간 주기로 최신 콘텐츠를 자동 수집한다. 사용자가 설정한 관심 태그와 직무 레벨을 기반으로 개인화된 피드를 제공하며, AI가 입문/주니어/미들/시니어 네 가지 레벨에 맞는 요약을 생성한다.

- **AI 기반 Q&A**

사용자가 기술 질문을 입력하면 AI가 질문을 먼저 명확하게 다듬고, FAISS 벡터 인덱스에서 관련 최신 문서를 검색해 RAG 방식으로 답변을 생성한다. 자주 반복되는 질문은 Redis에 캐싱하여 즉시 응답하고, 답변 생성 중에는 SSE 스트리밍으로 토큰이 실시간 출력된다.

- **커뮤니티**

질문과 답변을 커뮤니티에 게시하고, 채택된 우수 답변은 Q&A 모듈로 저장해 이후 유사 질문에 재사용된다. AI가 생성한 1차 답변과 커뮤니티 사용자의 답변이 함께 제공된다.

- **학습 히스토리 및 주간 리포트**

콘텐츠 열람, AI 요약 조회, 스크랩, 질문 작성, 게시글 작성 등 5가지 학습 행동이 자동 기록된다. 매주 학습 활동 통계와 AI 인사이트가 포함된 주간 리포트가 생성되며, 공유 링크로 외부에 공유할 수 있다.

• 구인구직 연계

축적된 학습 히스토리를 분석해 보유 기술 스택을 수치화하고, AI가 이력서 초안을 자동 생성한다. 사람 인 API를 통해 기술 스택에 맞는 채용 공고를 추천하고, 목표 회사별 맞춤 면접 예상 질문과 모범 답변을 제공한다.

• 최신 기술 동향

수집된 기술 문서를 FAISS 벡터 인덱스에 실시간 반영해 LLM의 지식 컷오프 이후 등장한 기술에 대해서도 정확한 Q&A가 가능하다. 태그 등장 빈도를 집계해 주간 트렌딩 기술을 산출하고, 시기별 트렌드 요약을 자동 생성한다.

다. 기존 서비스와의 차별점

구분	Stack Overflow	인프런/유데미	사람인/잡코리아	Trace
최신 기술 Q&A	커뮤니티 의존	없음	없음	RAG 기반 AI 자동 답변
레벨별 콘텐츠	없음	강의 레벨 구분	없음	AI가 동일 글을 레벨별로 재요약
학습 기록	없음	수강 이력	없음	5가지 행동 유형 자동 기록
취업 연계	없음	없음	공고 검색	학습 이력 → 이력서 → 면접 준비 자동 연결
최신 기술 동향	없음	없음	없음	1~6시간 주기 자동 수집 + 트렌드 요약

라. 프로젝트 기본 정보

프로젝트명	Trace: 콘텐츠 큐레이션 기반 개발자 성장 플랫폼
개발 기간	2026.04 ~ 2026.06 (캡스톤 기간, 약 3개월)
팀 구성	4명 (백엔드 2, AI 1, 프론트엔드 1)
기반 시스템	기존 Trace MVP (Epic A~F 구현 완료) 확장
대상 사용자	입문~시니어 개발자, 취업 준비 중인 개발자, 대학원생/연구원
개발 언어/프레임워크	Java 21 + Spring Boot (백엔드), Python 3.12 + FastAPI (AI), Next.js (프론트)

1.2 프로젝트 배경

가. 개발자 학습 생태계의 문제

현재 개발자들은 학습 과정에서 다음과 같은 근본적인 문제를 겪고 있다.

- **정보 분산**

개발 관련 콘텐츠가 Stack Overflow, Velog, Medium, GitHub, YouTube 등 수십 개의 플랫폼에 분산되어 있다. 하나의 기술 주제를 공부하기 위해 여러 플랫폼을 오가며 탭을 전환해야 하며, 플랫폼마다 계정과 북마크가 따로 관리된다. 원하는 정보를 찾기 위해 소비되는 탐색 시간이 실제 학습 시간보다 길어지는 현상이 발생한다.

- **학습 기록 미비**

개발자가 읽은 글, 해결한 문제, 스크랩한 자료가 개인 기록으로 체계적으로 남지 않는다. 6개월 전에 공부했던 내용을 다시 검색하거나, 이력서를 작성할 때 어떤 기술을 언제 공부했는지 기억하기 어렵다. 학습 이력이 성장의 증거로 활용되지 못하고 소모되는 구조다.

- **AI 시대 질문 방식 변화**

ChatGPT, Claude 등 생성형 AI의 등장으로 개발자들의 검색 및 질문 방식이 빠르게 변하고 있다. 그러나 일반 AI 챗봇은 개발 도메인에 특화된 컨텍스트가 부족하고, Stack Overflow처럼 커뮤니티 검증을 거친 답변을 제공하지 못한다.

- **성장 방향 불명확**

어떤 기술을 얼마나 학습했는지, 현재 내 수준이 어느 위치인지, 다음 단계로 무엇을 공부해야 하는지에 대한 가이드가 없다. 개인의 실제 학습 이력과 연계된 맞춤형 가이드는 제공되지 않는다.

- **레벨 맞춤 콘텐츠 부재**

동일한 주제라도 입문자와 시니어가 필요한 설명의 깊이와 전제 지식이 다르다. 현재 대부분의 플랫폼은 콘텐츠를 레벨별로 필터링하거나 재구성하는 기능을 제공하지 않는다.

나. 기존 LLM의 기술 최신성 한계

ChatGPT-5.1, Claude 4.6 Sonnet, Gemini 3.5 Pro 등 현재 가장 많이 사용되는 대형 언어 모델들은 학습 데이터 수집 시점, 즉 컷오프 날짜 이후에 등장하거나 업데이트된 기술에 대한 정보가 부정확하거나 존재하지 않는다.

Trace는 이 문제를 Stack Overflow, Velog, GitHub Trending, Hacker News 등의 소스에서 1~6시간 주기로 최신 기술 문서를 자동 수집하고, FAISS 벡터 인덱스에 실시간 반영하는 방식으로 해결한다. 사용자의 질문

이 들어오면 RAG 방식으로 최신 수집 문서를 컨텍스트로 첨부해 LLM에게 전달하여, 최신 기술에 대해서도 정확한 답변을 생성한다.

다. 학습-취업 연계 부재

현재 개발자 생태계에서 인프런, 유데미 같은 학습 플랫폼과 사람인, 잡코리아 같은 구인구직 플랫폼은 완전히 분리되어 있다. 개발자가 열심히 학습한 내용이 이력서나 취업 준비로 자동 연결되는 구조가 없다.

Trace에 축적된 학습 히스토리, 즉 읽은 기술 글, 작성한 질문, 채택된 답변, 스크랩한 콘텐츠는 사용자의 실질적인 기술 습득 증거다. 이를 AI로 분석하면 현재 보유 기술 스택과 깊이를 파악할 수 있고, 사람인 API의 채용 공고와 매칭하여 적합한 포지션을 추천하고, 목표 회사의 기술 스택에 맞는 면접 예상 질문을 자동 생성하는 것이 가능하다.

라. 타겟 사용자 선정 근거

Trace의 대상 사용자는 "개발자 성장 전 과정"이라는 플랫폼 목적에 따라 선정하였다. 레벨·직군을 불문하고 기술 학습이 필요한 모든 개발자가 해당되며, 아래 4개 세그먼트로 구체화한다.

타겟층	시장 규모 / 근거	핵심 페인포인트	Trace가 해결하는 방식
입문~주니어 개발자	국내 코딩 부트캠프 연간 수료자 약 1만 명 이상, 컴퓨터 관련 학과 졸업생 지속 증가	레벨에 맞지 않는 정보 과부하, 학습 방향 불명확	레벨별 AI 요약(입문/주니어/미들/시니어), 학습 히스토리 기반 성장 리포트
취업 준비 중인 개발자	개발 직군 채용 공고 연간 증가 추세, 부트캠프·전공자 모두 해당	학습 이력과 이력서·면접 준비가 완전히 단절됨	학습 → 이력서 자동 생성 → 회사별 면접 Q&A 자동 연결 (Epic G)
미들~시니어 개발자	Stack Overflow Developer Survey 2024: 응답자의 약 65%가 5년 이상 경력	범용 LLM의 지식 컷오프로 최신 기술 답변이 부정확	RAG 기반 최신 기술 실시간 반영 Q&A (Epic H)
대학원생·연구원	국내 대학원 재학생 약 33만 명(2024년 교육통계서비스)	기술 문서 기반 학습에도 커뮤니티·취업 연계 경로 부재	하이프레인넷(연구직) 연동, 연구노트·Q&A 모듈화 (Epic G/I)

선정 원칙: 기존 서비스는 "학습" 또는 "취업" 중 하나에만 집중한다. Trace는 학습 이력이 취업으로 자동 연결되는 end-to-end 구조이므로, 취업 준비 단계에 있는 모든 레벨의 개발자를 동시에 타겟으로 삼는 것이 합리적이다.

1.3 프로젝트 목표

가. RAG 기반 최신 기술 동향 반영 Q&A 시스템 구축

Stack Overflow API, Velog GraphQL, GitHub RSS, Hacker News API 등 5개 이상의 소스에서 소스별로 차등화된 주기(1~6시간)로 최신 기술 콘텐츠를 자동 수집한다. 수집된 문서는 800자 이상의 본문이 있는 경우에만 전처리(HTML 파싱, 텍스트 정규화, 중복 URL 제거)를 거쳐 FAISS 벡터 인덱스에 추가된다.

사용자가 질문을 입력하면 시스템은 먼저 Redis에서 임베딩 코사인 유사도 0.9 이상인 기존 답변을 탐색(캐시 히트 시 100ms 이내 응답)하고, 캐시 미스 시 FAISS 인덱스에서 관련 문서 상위 5개를 검색해 Claude API의 컨텍스트로 제공한다. 이를 통해 학습 데이터 컷오프 이후 기술에 대해서도 정확한 답변을 생성한다.

목표 성능 지표:

- 캐시 히트율: 반복 질문 30% 이상에서 캐시 응답
- AI 응답 시간: 첫 토큰 출력 3 초 이내 (SSE 스트리밍)
- FAISS 인덱스 업데이트 주기: 핵심 소스(SO, Velog) 2 시간 이내

나. 학습 히스토리 기반 취업 연계 파이프라인 구축

사용자의 학습 활동(콘텐츠 열람 · AI 요약 조회 · 스크랩 · 질문 작성 · 답변 채택)을 PostgreSQL history 테이블에 누적한다. 축적된 이력을 분석해 기술별 학습 빈도와 깊이를 수치화하고, Claude API를 통해 JSON Schema 기반 구조화 이력서 초안을 자동 생성한다.

사람인 Open API(1회 최대 500개 공고)와 연동해 보유 기술 스택·직무 레벨에 맞는 채용 공고를 매칭한다. 사용자가 목표 회사를 선택하면 해당 공고의 요구 기술을 추출하고, Claude API로 회사 맞춤 면접 예상 질문 10개와 모범 답변을 생성한다. 학습 이력에 없는 요구 기술은 Trace 콘텐츠 · YouTube 영상 · 서적 추천으로 연결해 역량 보완 경로를 제시한다.

다. Q&A 모듈화 재사용 시스템(연구노트) 개발

채택된 답변, AI 생성 답변, 사용자가 직접 저장한 Q&A 쌍을 "지식 모듈"로 저장한다. 각 모듈은 질문 임베딩 벡터와 함께 저장되어 유사 질문 입력 시 FAISS 유사도 검색으로 우선 참조되며, 동일 또는 유사한 질문이 반복될 때 API 재호출 없이 즉시 응답하여 응답 속도를 높이고 AI API 비용을 절감한다.

개인 연구노트 뷰에서 저장된 모듈을 태그별·날짜별로 조회할 수 있으며, 일정 기간이 지난 항목에 대해 복습 알림과 퀴즈를 제공한다.

라. 실시간 기술 동향 수집 자동화 워크플로우 구현

Spring Batch + FastAPI 스케줄러를 연동해 아래 전체 파이프라인을 자동화한다.

- 크롤링: 소스별 차등 주기(SO 1h · Velog 2h · GitHub RSS 6h)로 5 개 소스에서 병렬 수집
- 전처리: HTML 파싱 → 800 자 미만 필터링 → canonical_url 중복 제거 → UPSERT
- 태그 자동 분류: AI 가 키워드를 추출해 기존 태그와 매칭하거나 신규 태그를 자동 생성
- FAISS 인덱싱: 신규 문서를 임베딩 벡터로 변환해 실시간 인덱스에 추가
- 트렌드 요약: 매주 월요일 새벽 3 시, 트렌딩 태그 TOP 10 산출 + Claude API 로 주간 기술 동향 자연어 요약 생성 → 기존 주간 리포트(Epic F)와 연계해 사용자별 맞춤 제공

2. 프로젝트 개요

2.1 프로젝트 설명

Trace는 기존 MVP(1단계)를 기반으로 캡스톤 기간에 3개의 신규 Epic을 추가하는 2단계 방식으로 개발된다.

1 단계 (완료): Trace MVP (Epic A~F)

기존에 구현된 핵심 기능으로, 본 캡스톤 프로젝트의 기반이 된다. Spring Boot 백엔드 API, FastAPI AI 서버, Next.js 프론트엔드의 3-tier 구조로 구현되었으며, PostgreSQL(구조화 데이터), MongoDB(AI 결과물), Redis(캐시/세션)를 병행 사용한다.

Epic	기능	상태
A. 회원/프로필	이메일+소셜(GitHub/Google) 로그인, 관심태그/직무/레벨 설정	완료
A+. 포인트/배지	학습 행동별 포인트 적립, 배지 잠금 해제, 누적 포인트/스트릭 조회	완료 (DP-269)
B. 콘텐츠 피드	태그 기반 개인화 피드, 글 상세, 스크랩/좋아요, 검색	완료
C. AI 요약	레벨별 AI 요약 (주니어/미들/시니어), Redis+MongoDB 3단계 캐시	완료
C+. AI 퀴즈	콘텐츠 기반 레벨별 퀴즈 생성, 통과 시 포인트 적립, Redis+MongoDB 3단계 캐시	완료
D. Q&A/커뮤니티	AI 질문 개선, 커뮤니티 게시, AI 1차 답변, 답변 채택	완료
E. 학습 히스토리	5가지 행동 유형 기록(content_opened, scrapped 등), actionTypes 필터 조회	완료
F. 주간 리포트	활동 집계, 바/레이더 차트, AI 인사이트, 공유 링크 생성	완료

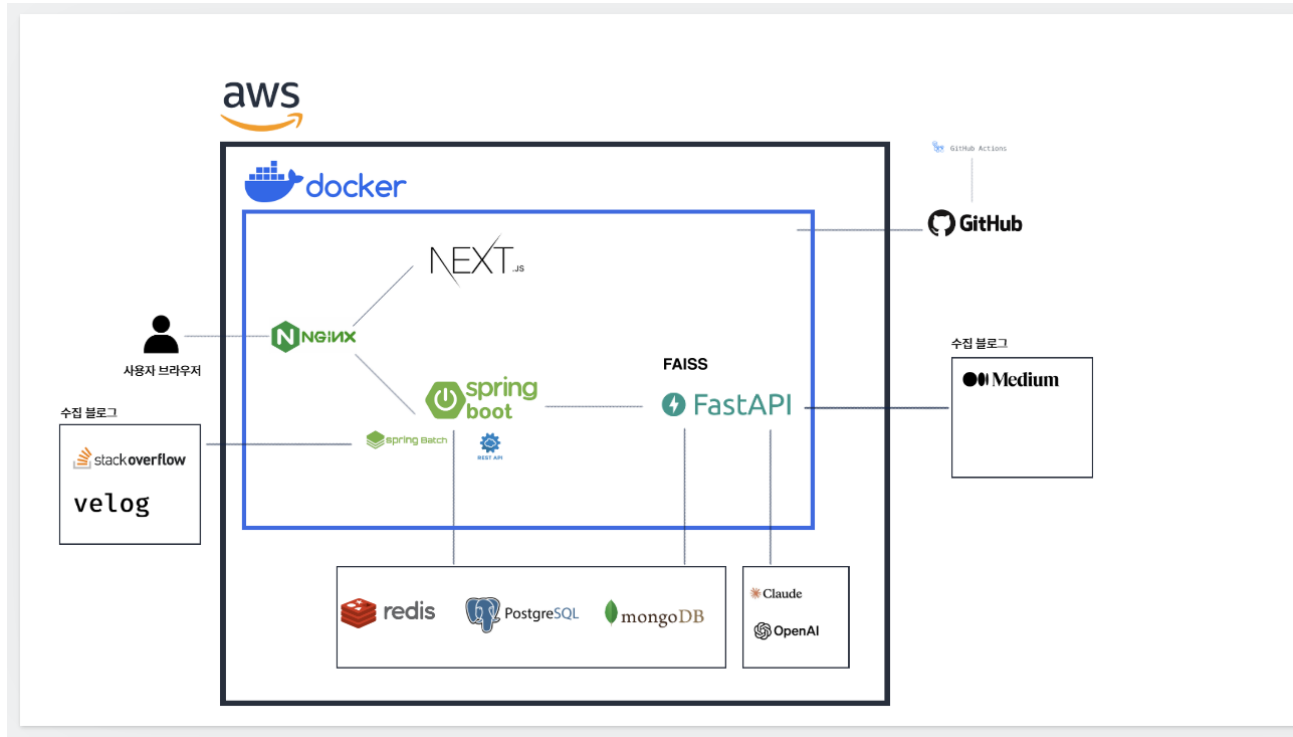
2 단계 (캡스톤 기간 개발): 확장 Epic G~I

Epic	핵심 기능	기술적 차별점
G. 구인구직 연동	사람인 API 매칭, 이력서 자동 생성, 회사별 면접 Q&A, 하이브레인넷, 공모전 정보	학습 이력 → 취업 자동 연결
H. 최신 기술 동향	5개 소스 실시간 크롤링, FAISS 자동 재인덱싱, 트렌드 요약	LLM 지식 컷오프 문제 해결
I. 연구노트/퀴즈	Q&A 모듈 저장·재사용, 복습 퀴즈, 로드맵 추천, AI 답변 대기 UX	커뮤니티 지식 자산화

2.2 프로젝트 구조

가. 시스템 아키텍처

3-tier 구조: 사용자 브라우저 → Nginx (SSL 중단, 리버스 프록시) → Next.js (프론트, :3000) + Spring Boot (REST API 서버, :8080) → PostgreSQL / Redis / MongoDB → FastAPI AI Server (:8000) → FAISS / Claude API / 크롤러

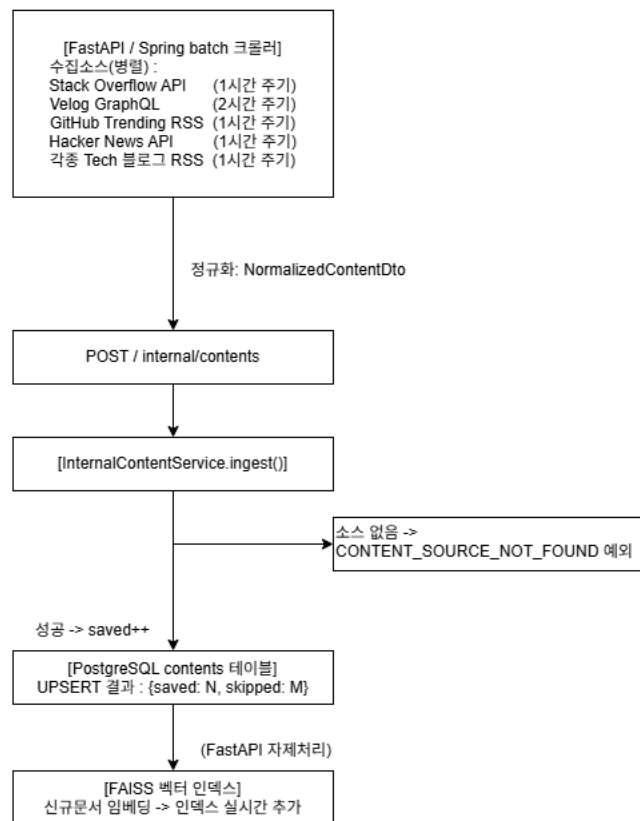


각 서버의 역할:

- Nginx: SSL 인증서 적용, 도메인별 라우팅 (api.trace.kr → Spring Boot, trace.kr → Next.js)
- Spring Boot: 비즈니스 로직 처리, JWT 인증, FastAPI 와의 내부 REST 통신
- FastAPI: 크롤링 스케줄, RAG 파이프라인, Claude API 호출, FAISS 인덱스 관리
- PostgreSQL: 사용자/콘텐츠/커뮤니티/히스토리/리포트 구조화 데이터 저장
- MongoDB: AI 요약 결과(ai_summaries), 주간 리포트 인사이트, 이벤트 로그 저장
- Redis: JWT Refresh Token, OAuth state, AI 요약 캐시(TTL 7 일), Q&A 캐시(TTL 7 일)

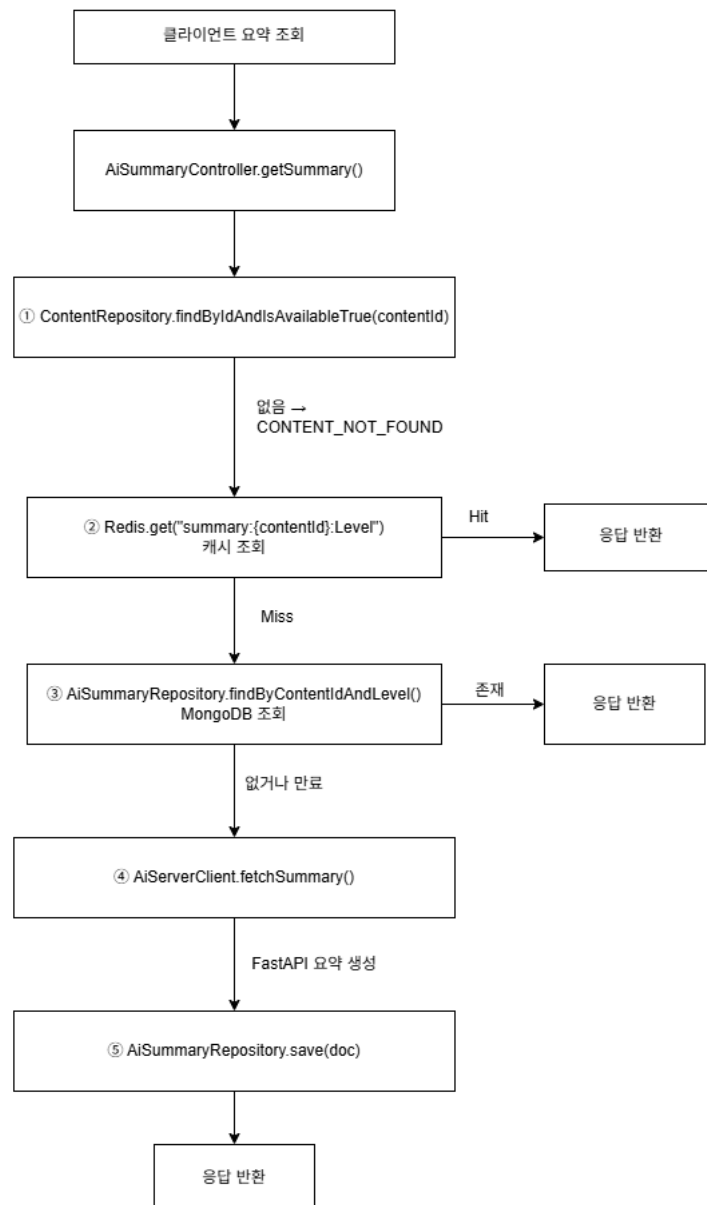
나. 데이터 흐름

콘텐츠 수집 파이프라인 (Epic H): 소스별 수집기(FastAPI Scheduler)가 Stack Overflow API(1시간), Velog GraphQL(2시간), GitHub Trending RSS(6시간), Hacker News API(1시간), 기술 블로그 RSS(6시간) 주기로 병렬 수집한다. 수집 후 HTML 파싱 → 본문 텍스트 추출 → 800자 미만 필터링 → canonical_url 중복 확인 → PostgreSQL(contents 테이블 UPSERT) + FAISS 인덱스 실시간 업데이트 순으로 처리된다.



AI Q&A 플로우 (Epic D + H):

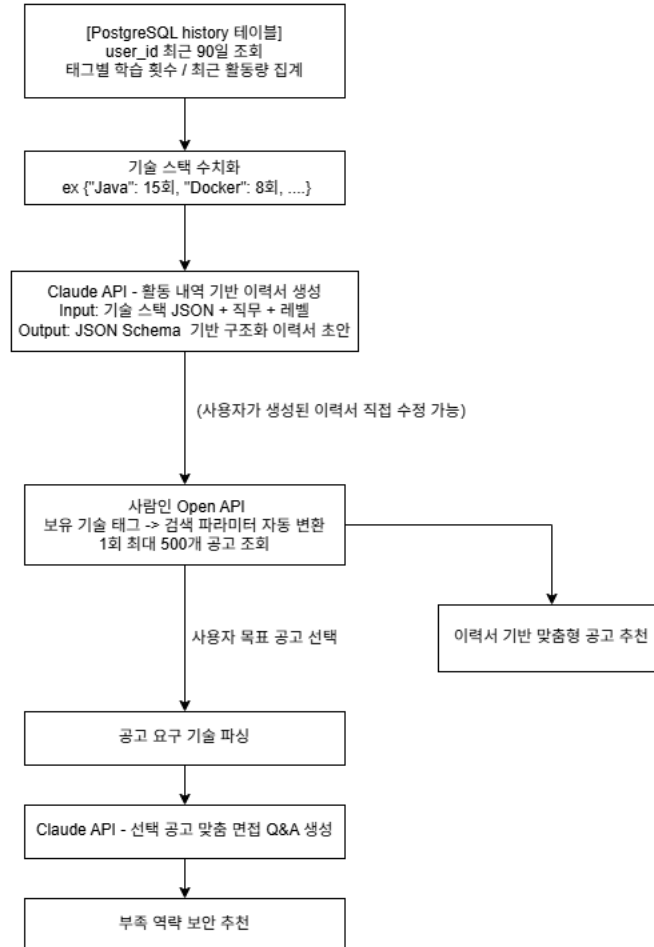
- [1 단계] Redis 캐시 조회: 질문 임베딩 생성 → 코사인 유사도 검색 → 유사도 > 0.9 시 캐시 히트(100ms 이내 즉시 응답)
- [2 단계] FAISS RAG 검색 (캐시 미스 시): FAISS 인덱스에서 최신 수집 문서 상위 5 개 벡터 검색
- [3 단계] Claude API 호출: 시스템 프롬프트 + RAG 컨텍스트 5 개 + 사용자 질문 → SSE 스트리밍으로 첫 토큰 3 초 이내 출력
- [4 단계] 결과 저장: Redis Q&A 캐시(TTL 7 일) + MongoDB event_logs + 채택 시 Q&A 모듈 저장(Epic I)



구인구직 연계 파이프라인 (Epic G):

- 히스토리 분석: PostgreSQL history 테이블에서 기술 태그별 학습 횟수·최근 30 일 활동량 수치화
- 이력서 자동 생성: 학습 통계 JSON + 직무/레벨 입력 → Claude API → 구조화 이력서(JSON Schema) 출력, 사용자 직접 수정 가능
- 사람인 공고 매칭: 보유 기술 태그 → 검색 파라미터 자동 변환 → 1 회 최대 500 개 공고 조회 → 기술 일치율 스코어 상위 20 개 추천
- 면접 Q&A 생성: 목표 공고 선택 → 요구 기술 스택 파싱 → Claude API → 맞춤 면접 예상 질문 10 개 + 모범 답변 + 부족 역량 분석

- 역량 보완 추천: Trace 최신 콘텐츠 상위 5 개 + YouTube 강의 영상 3 개 + 직무별 분기(회사원/연구원/대학원생)



2.3 시나리오

가. 최신 기술 학습 시나리오

주니어 백엔드 개발자 A가 Java 가상 스레드(Virtual Threads)에 대해 공부하는 상황:

- Trace 개인화 피드에서 "Spring Boot 3.2 Virtual Threads 적용기" Velog 포스트를 발견 (2 시간 전 수집)
- "주니어" 레벨 요약 선택 → Redis 캐시 조회 → 캐시 미스 시 FastAPI 가 RAG 컨텍스트와 함께 Claude API 호출 → 결과 Redis 에 7 일간 캐시
- Q&A 탭에서 질문 입력 → AI 가 질문을 "Java 21 Virtual Thread 와 Platform Thread 의 메모리 구조 차이 및 성능 비교"로 개선

- FAISS 인덱스에서 JDK 21 릴리즈 노트, Stack Overflow 관련 답변 등 5 개 문서가 RAG 컨텍스트로 활용되어 최신 벤치마크 수치가 포함된 정확한 답변 생성 (SSE 스트리밍)
- Q&A 쌍을 연구노트에 저장 → 2 주 후 복습 알림 수신

나. 구인구직 연계 시나리오

취업 준비 중인 주니어 백엔드 개발자 B가 카카오 백엔드 엔지니어 포지션에 지원하는 상황:

- 최근 90 일 학습 히스토리 분석 (Spring Boot 22 회, Java 15 회, Docker/Kubernetes 8 회, Redis 5 회 열람)
- AI 가 이력서 초안 자동 생성: Java(상), Spring Boot(상), Docker(중), Redis(하)
- 사람인 API 가 스택 기반으로 백엔드 공고 500 개 조회 → 기술 일치율 상위 20 개 추천
- "카카오 - 서버 플랫폼 개발" 공고 선택 → 요구 기술(Kotlin, Spring, Kafka, MySQL, Kubernetes) 분석 → Kotlin, Kafka 역량 부족 파악
- Claude API 가 카카오 맞춤 면접 예상 질문 10 개 생성 + 부족 역량(Kafka)에 대해 Trace 최신 글 3 개 및 YouTube 영상 2 개 추천

다. 기술 동향 자동화 시나리오

시스템이 새벽 시간대에 최신 기술 정보를 자동으로 수집하고 반영하는 상황:

- 매일 새벽 2 시, FastAPI 스케줄러가 소스별 수집기를 병렬로 실행 (SO, Velog, GitHub Trending 등)
- 수집 원시 데이터가 전처리 파이프라인 통과: HTML 태그 제거 → 본문 800 자 미만 필터링 → 중복 URL 확인
- AI 가 콘텐츠에서 키워드 추출 → 기존 태그 매핑 또는 신규 태그 자동 생성 → 최근 7 일 태그 등장 빈도 집계 → 트렌딩 태그 TOP 10 갱신
- 새 문서들이 임베딩 벡터로 변환되어 FAISS 인덱스에 추가 → 이후 사용자 질문 시 RAG 컨텍스트로 활용
- 매주 월요일 새벽 3 시, 주간 트렌드 요약 배치 실행 → Claude API 로 개발 트렌드 자연어 리포트 생성 → 기존 주간 리포트(Epic F)에 트렌드 섹션 추가

2.4 기대효과

기대효과	측정 지표	목표값
AI API 비용 절감	Redis 캐시 히트율	반복 질문의 30% 이상 캐시 응답
AI 응답 시간 단축	첫 토큰 출력까지 소요 시간	SSE 스트리밍 기준 3초 이내

최신 기술 정보 정확도 향상	FAISS 인덱스 문서 수	주당 신규 문서 500개 이상 반영
취업 준비 효율화	이력서 생성 소요 시간	수동 작성 대비 초안 생성 시간 90% 단축
커뮤니티 지식 재사용	Q&A 모듈 재참조율	신규 질문 중 20% 이상 기존 모듈 히트
빌드 안정성	CI 빌드 성공률	99% 이상 (SonarCloud Quality Gate 통과 기준)

2.5 제약조건

가. AI API 비용 및 응답 속도

Claude API는 입력/출력 토큰 수 기반 과금 모델이다. 캡스톤 기간 사용량 목표는 월 \$10~50 수준으로 관리한다. 이를 위해 다음 4가지 전략을 적용한다.

- Redis Q&A 캐싱: 반복 질문 API 재호출 차단
- Anthropic Prompt Caching: 긴 시스템 프롬프트 재사용
- max_tokens 2,048 제한: 응답 길이 상한 설정
- temperature 0 고정: 결정적 응답으로 캐시 효율 최대화

Ollama 기반 로컬 모델은 GPU 서버가 없는 환경에서 Llama3 7B 기준 응답에 15~30초가 소요되어 실시간 데모가 불가능하다. 따라서 캡스톤 기간에는 Claude API를 사용하고, 이후 GPU 서버 확보 시 로컬 모델 전환을 검토한다.

나. 외부 API 제약

- 사람인 Open API: 1 회 요청 최대 500 개 공고, 일일 요청 횟수 제한 존재. 공고 데이터를 Redis 에 1 시간 TTL 로 캐싱하여 중복 API 호출을 방지한다.
- YouTube Data API v3: 무료 티어 일일 10,000 유닛 쿼터. 영상 검색 1 회가 약 100 유닛이므로 일 최대 100 회 검색 가능. 검색 결과를 Redis 에 24 시간 TTL 로 캐싱한다.
- Stack Overflow API: 인증 없이 하루 300 회, 인증 시 10,000 회 요청 가능. API 키 등록 후 사용한다.

다. 크롤링 법적 제약

수집 대상 사이트의 robots.txt 및 이용약관을 반드시 준수한다. Stack Overflow는 CC BY-SA 4.0 라이선스로 공식 API를 통한 데이터 활용이 허용된다. Velog는 GraphQL 공개 API를 사용한다. 직접 크롤링이 필요한 경우 크롤링 딜레이(1~2초)를 적용하고, 서버 부하를 유발하지 않는 범위에서 수행한다.

라. 기간 및 인력 제약

캡스톤 기간(3개월) 내 Epic G~I 전체 구현은 현실적으로 어렵다. MoSCoW 우선순위를 적용해 다음과 같이 범위를 조정한다.

- Must (캡스톤 기간 내 반드시 완료): 사람인 API 연동, 이력서 자동 생성, 회사별 면접 Q&A, 실시간 크롤링 파이프라인, FAISS RAG 업데이트
- Could (여유 시 추가): YouTube 강의 추천, 하이브레인넷 연동, 공모전 정보 크롤링, AI 답변 대기 UX 개선

콘텐츠 기반 AI 퀴즈(GET /contents/{id}/quiz)는 MVP C+ 항목으로 선구현 완료됨.

2.6 관련기술

기술	설명	Trace에서의 구체적 활용
RAG	LLM 호출 전 외부 문서를 벡터 검색으로 찾아 컨텍스트로 첨부하는 기법. LLM의 지식 한계를 외부 데이터베이스로 보완한다.	사용자 질문 입력 시 FAISS 인덱스에서 관련 최신 문서 상위 5개를 검색해 Claude API 시스템 프롬프트에 추가. 컷오프 이후 기술 정보를 정확히 답변 가능하게 한다.
FAISS	Facebook AI Research가 개발한 고속 벡터 유사도 검색 라이브러리. 수백만 개 벡터를 밀리초 내에 검색한다.	수집된 기술 문서 및 Q&A 쌍을 임베딩 벡터로 저장. 질문 임베딩과의 코사인 유사도를 계산해 관련 문서를 실시간 검색.
LangChain	LLM 기반 파이프라인 구성을 위한 Python 프레임워크.	FastAPI AI 서버에서 RAG 파이프라인 구성 (문서 로딩 → 청킹 → 임베딩 → FAISS 저장 → 검색 → Claude 호출).
SSE	HTTP 기반 서버→클라이언트 단방향 스트리밍 프로토콜. WebSocket보다 구현이 단순하고 HTTP/2와 호환된다.	Claude API의 스트리밍 응답을 Spring Boot에서 SseEmitter로 브라우저에 토큰 단위로 전달.
Spring Batch	Spring 기반 대용량 배치 처리 프레임워크. Job, Step, Reader/Processor/Writer 구조로 안정적인 배치를 구현한다.	주간 리포트 생성(매주 월요일 새벽), 트렌드 요약 생성, 만료 캐시 정리 등 정기 배치 작업.
사람인 Open API	사람인 채용 공고 검색 REST API. 직무/기술/지역/경력 등 다양한 필터를 지원한다.	사용자 보유 기술 태그를 검색 파라미터로 변환해 맞춤 공고를 조회. 공고 요구 기술과 사용자 기술의 일치율을 스코어링해 추천 순위 결정.
Prompt Engineering	LLM에게 효과적인 입력을 설계하는 기법. 시스템 프롬프트, Few-shot 예시, Tool Use 등을 활용한다.	레벨별 요약, 면접 질문 생성, 이력서 작성, 질문 개선 등 각 기능별 특화 프롬프트를 설계. Tool Use로 JSON 출력을 강제해 파싱 성공률 100% 달성.

YouTube Data API v3	YouTube 영상 검색 및 메타데이터 조회 REST API.	면접 준비 중 부족한 기술 역량에 대해 관련 강의 영상을 검색해 추천. 일일 10,000 유닛 쿼터 초과 방지를 위해 Redis 24시간 캐시 적용.
---------------------	------------------------------------	---

2.7 개발도구

구분	기술	버전	비고
백엔드	Spring Boot	3.5.11	
	Java	21 (LTS)	가상 스레드 지원
	JPA/Hibernate	-	
	QueryDSL	5.1.0:jakarta	동적 쿼리
	Gradle	최신	빌드 도구
AI 서버	FastAPI	최신	비동기 REST
	Python	3.12	
	LangChain	최신	RAG 파이프라인
	FAISS	최신	벡터 인덱스
	Claude API	claude-sonnet-4-6	Anthropic
프론트엔드	Next.js	최신	App Router
	React	18+	
	TypeScript	최신	
데이터베이스	PostgreSQL	16 (AWS RDS)	구조화 데이터
	MongoDB	7	AI 결과물, 이벤트
	Redis	7	캐시, 세션
인프라	Docker	최신	컨테이너화
	AWS EC2	-	서버 호스팅
	Nginx	최신	리버스 프록시, SSL
	GitHub Actions	-	CI/CD 자동화
품질 관리	SonarCloud	-	신규 코드 커버리지 80% 이상
	JaCoCo	0.8.12	테스트 커버리지 측정

	JUnit 5 + Mockito	-	단위/통합 테스트
협업	Jira	Cloud	이슈 170건(Epic 14·Task 128·Story 19·Spike 7·Bug 2), 칸반 워크플로 4단계, 완료율 77%
	Confluence	Cloud	64페이지: ADR 12건, 회의록 12건, 트러블슈팅 6건, API 명세 4건
	GitHub	-	소스 코드 관리, PR 리뷰, Jira 티켓 자동 연동

3. 프로젝트 추진 체계 및 일정

3.1 역할 분담

이름	역할	주요 담당 업무
홍근	PM / 백엔드 리드	전체 시스템 아키텍처 설계 및 유지 관리, Epic G(구인구직) API 개발(사람인 연동, 이력서 자동 생성, 면접 Q&A), CI/CD 파이프라인 관리(GitHub Actions, SonarCloud Quality Gate), 스프린트 플래닝 및 일정 조율, Jira 이슈 체계 설계(14 Epic · 4단계 워크플로) 및 Confluence 지식 베이스 운영(64페이지 · ADR 12건 · 회의 템플릿 4종), ML / AI 테크리드
하영	백엔드 서브	Epic H(기술 동향) 배치 시스템 개발, 콘텐츠 수집 파이프라인 확장(크롤링 소스 추가, 주기 최적화), 크롤링 법적 검토(robots.txt, 이용약관), Spring Batch 스케줄러 설계
수현	AI 엔지니어	RAG 파이프라인 구축 및 FAISS 인덱스 관리, 기능별 특화 프롬프트 엔지니어링(Q&A, 이력서 생성, 면접 질문, 동향 요약), AI 응답 속도 최적화(캐싱, Prompt Caching), Golden Set 기반 품질 평가(정확도, JSON 파싱 성공률)
보민	프론트엔드	구인구직/기술 동향/연구노트 화면 UI/UX 설계 및 구현, SSE 기반 AI 답변 스트리밍 실시간 렌더링, 사람인 공고 카드 UI, 이력서 편집기 구현, 반응형 디자인

3.2 작업 흐름도

개발 워크플로우:

- Jira 티켓 생성 (To Do)
- 브랜치 생성: Claude Code 자동화(auto/feature/DP-{번호}) 또는 직접 작업(feature/DP-{번호}-{기능명})
- 개발 (In Progress): 커밋 메시지 형식 "DP-{번호}: {작업 내용}", 새 Service/Controller 마다 단위 테스트 작성
- PR 생성 → 팀원 코드 리뷰: PR 제목 "[DP-{번호}] {설명}", Jira 링크 자동 삽입
- GitHub Actions CI 자동 실행: Build & Test(빌드+테스트) + SonarCloud Quality Gate(커버리지 80%, 코드 중복 3%, Security 0) + auto/ 브랜치 자동 머지 설정
- auto/ 브랜치: develop 에 자동 squash 머지 / feature/ 브랜치: 팀원 확인 후 수동 머지
- Jira 티켓 Done 처리 + 주간 보고서 반영

지식 관리 흐름 (Jira · GitHub · Confluence 삼각 구조): "무엇을 할지(Jira) → 어떻게 구현했는지(PR) → 왜 그렇게 결정했는지(Confluence)"의 구조로 프로젝트 전 과정을 추적한다.

활동	Confluence 산출물	Jira 연동 방식
아키텍처 의사결정	ADR 12건 (DB 분리, JWT, API 포맷, AI JSON 스키마, 캐시 전략 등)	Spike 티켓에 Confluence 링크 첨부
정기 미팅 4종	회의록 12건 (데일리 10분 · 위클리 플래닝 45분 · 데모/리뷰 30분 · 회고 30분)	회고 Try 항목 → 다음 스프린트 Jira 티켓 전환
장애 해결	트러블슈팅 6건 (명명: [도메인] 설명 (Jira 티켓))	Jira 티켓 ↔ Confluence 문서 양방향 링크
API 설계	API 명세 4건 (초안 → 수정 → AI 내부 API → Swagger 설정)	API 변경 시 관련 Jira 태스크 생성
팀 프로세스	팀 헌장, 협업 운영 가이드, 온보딩 3종	신규 합류 시 온보딩 가이드로 자가 셋업

스프린트 운영 방식

- 주기: 1~2 주 단위 스프린트
- 플래닝: 스프린트 시작 전 월요일 정기 미팅 (13~14 시)에서 티켓 할당 및 예상 공수 산정
- 일일 현황: Jira 칸반 보드 (To Do → In Progress → In Review → Done) 실시간 업데이트
- 회고: 스프린트 종료 후 KPT(Keep/Problem/Try) 회고 미팅 → 결과 Confluence 기록, Try 항목을 다음 스프린트 Jira 티켓으로 전환
- 산출물: 모든 설계 결정은 Confluence ADR 로 문서화, 트러블슈팅은 Jira 티켓과 연동된 Confluence 문서로 기록

3.3 개발 일정

가. 전체 WBS

기간	마일스톤	주요 작업	담당
2026.03 1주	프로젝트 착수	교수님 킥오프 미팅, 주제 확정, 역할 분담, 요구사항 정의	전체
2026.03 2~3주	설계 + MVP 개발 착수	시스템 아키텍처 설계, DB 스키마, API 명세 초안, 개발 환경 구성, Sprint 0	전체
2026.03 4주	MVP 완료 + 제안서 제출	인증/콘텐츠/커뮤니티/리포트/히스토리/포인트 도메인 구현, 제안서 최종 제출(3/31)	홍근/하영
2026.04 1주	인프라 구축 완료	AWS EC2 세팅, RDS PostgreSQL, SSL 인증서, Nginx 설정, CD 파이프라인 완성	홍근
2026.04 2~4주	Epic G 개발	사람인 API 연동, 히스토리 분석, 이력서 자동	홍근/수현

		생성, 면접 Q&A 생성	
2026.05 1~2주	Epic H 개발	크롤링 소스 확장, FAISS 자동 재인덱싱, 트렌딩 태그 집계, 트렌드 배치	하영/수현
2026.05 3~4주	Epic I 개발 + 통합 테스트	Q&A 모듈화 저장, 연구노트 뷰, E2E 시나리오 테스트, 성능 테스트	전체
2026.06 1주 (~6/4)	최종 발표 준비 + 발표전 시회	테스트 결과 기반 수정, 3개 시나리오 데모 준비, 발표 자료 완성, 발표전시회(6/4)	전체

나. 주간 보고서 일정 (2 주차부터, 1 주차 시작: 2026-03-09)

주차	날짜	보고 내용
2주차	2026.03.16	교수님 킥오프 미팅 완료, 주제 확정, 역할 분담, 팀 협업 환경 구성
3주차	2026.03.23	요구사항 정의 완료, 시스템 아키텍처 · DB 스키마 설계, API 명세 초안
4주차	2026.03.30	MVP 개발 착수 (Sprint 0 · 인증/사용자 도메인), 제안서 제출(3/31)
5주차	2026.04.06	콘텐츠 · 커뮤니티 도메인 완료, 리포트/히스토리 도메인 착수
6주차	2026.04.13	MVP 완료 (리포트 · 히스토리 · 포인트 도메인), 인프라 구축 완료
7주차	2026.04.20	Epic G 핵심 개발 (사람인 API 연동, 학습 히스토리 분석 로직)
8주차	2026.04.27	Epic G 완료 (이력서 자동 생성, 면접 Q&A 생성), Epic H 착수
9주차	2026.05.04	Epic H 개발 (크롤링 소스 확장, FAISS 자동 재인덱싱, 트렌딩 태그)
10주차	2026.05.11	Epic H 완료 (트렌드 요약 배치), Epic I 개발 착수
11주차	2026.05.18	Epic I 진행 현황 (Q&A 모듈화 저장, 연구노트 뷰)
12주차	2026.05.25	통합 테스트 실행 결과 (E2E · 성능 테스트), 버그 리스트
13주차	2026.06.01	버그 수정 완료, 최종 데모 시나리오 확정, 발표 자료 작성
14주차	2026.06.08	최종 발표전시회(6/4) 완료, 사후 정리

4. 참고자료

번호	자료	출처
[1]	사람인 Open API 가이드	https://oapi.saramin.co.kr/guide
[2]	LangChain 공식 문서	https://python.langchain.com
[3]	FAISS 공식 문서	https://faiss.ai
[4]	YouTube Data API v3	https://developers.google.com/youtube/v3
[5]	Stack Overflow API v2.3	https://api.stackexchange.com
[6]	Anthropic Claude API 문서	https://docs.anthropic.com
[7]	RAG 논문: "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"	Facebook AI Research, 2020
[8]	Spring Batch 공식 문서	https://spring.io/projects/spring-batch